



Bilkent University

Department Of Computer Engineering

Bilkent Information Office Web Page

CS-319 Section 1

Group 2

Deliverable-3 2nd Iteration

19.12.2024

Instructor: Eray Tüzün

Teaching Assistant: Yahya Elnouby

Group Members:

Name	ID
Ayça Candan Ataç	22203501
Elif Ceren Çelik	22202534
Elif Ece Can	22201800
Emre Furkan Akyol	22103352
Mustafa Özkan İr	22103267
Ahmet Yağız Sarıdoğan	22202072

Top-2 Design Goals

The design goals stem from the analysis of non-functional requirements, which aim to make the software more viable. While designing the project, these two design goals will be prioritized, even if potential trade-offs occur, which will be noted in this report.

1. Usability

- The core design goal of the Information Office Web Page will be usability. The website should be easy to use since the web application is designed to introduce our university to high schools, prospective students, and their parents. We specified our target design to aim that users who use the app to obtain information about the university won't be confused while navigating our website. To achieve this, we will enhance navigation by incorporating clickable headings alongside relevant icons. This design approach ensures the website is more intuitive, as the links to information will be immediately apparent to users. After clicking, users will be able to navigate to the part where they want to get additional details from the website in less than 2 seconds. The main page design of the Information Office Web Page will cover all relevant details about the visiting appointment, seminars, FAQ, the university booklet, and terms and conditions. While obtaining all these options, we will use a table layout and oriented margins to each point so that it can easily be noticeable by the users. In addition, we will provide the relevant details immediately after that click, ensuring that the user can see all important topics that they search for within one or two clicks from the main page.
- Another point of the usability goal is that the director and Information Office members can easily adapt to the usage of our app. We will implement a dynamic website for them to prioritize their main responsibilities with ease. For instance, for each user type (Guide, advisor, head secretary, director), the dashboard will differ according to their main duties, including the to-do list that they can check and update after every login. The sections on the dashboard page will be clickable, allowing users to navigate the detailed page of the clicked entry (The tour section will redirect to the tour page). Also, the website contains categories to click on from the left side of the page, such as tours, guide schedules, high school analytics, and profile pages. Thus, the website lowers the possible confusion that Information Office members could experience in the first few days after starting to use the website.
- Lastly, managing the tour operations quickly will increase the performance of the information office team members since the most time-consuming part is managing the tours. To provide this, the website will rely on a notification system that performs by alerting all related users within ~1 second. This enhances usability by informing information office members of new updates about upcoming tours. To catch their

attention, we will also provide a red dot on the notification icon to help them keep up with the new updates.

2. Reliability

- The second important design goal of the Information Office Web Page will be reliability. The website must consistently deliver accurate information about the tour operations and perform important actions without errors, such as assigning a guide to a tour or canceling an event. In order to achieve this, we will consider lots of corner cases, such as not assigning the guide to the tour if he/she has already been assigned to another one in that time period. Another corner case is allowing an advisor to accept or reject a tour application only if it is scheduled on their designated responsibility day. We used the Google Sheet Integration test to ensure events were correctly fetched to our database after a counselor submitted the form.
- User data, tour schedules, and other critically sensitive information about the application will be securely stored using PostgreSQL, with encryption and access control mechanisms in place. To maintain the reliability of our app during event processing, we implemented role-based access control (RBAC) to restrict access to event tables based on user roles. Additionally, we integrated a JWT-based authorization system in Spring Boot to ensure secure access by validating user identities and permissions for API requests. For example, only an advisor can respond to tour application requests and will see a different tour schedule table compared to the guide's access. The same applies to the Coordinator's screen for fair applications compared to the guide's screen.
- Updates to the database will occur in real-time: for tour applications, the Google Sheet API will fetch new applications within 10 seconds. For user profile operations, we maintain a latency of less than 1 second, ensuring that users always have access to the latest information.

3. Possible Trade-offs

3.1. Usability vs. Performance

Ensuring the website is highly usable will force us to use additional resources, such as loading visual elements for the detailed dashboards and structural layouts. While this improves user experience, it could slightly increase page load times since this heavy design can lead to higher memory consumption and slower startup times compared to lightweight frameworks.

3.2. Usability vs. Security

For the sake of usability, we will not be planning to implement a login page for prospective students and counselors. Although this eases user experience, our

app can encounter some security issues, such as fake appliances for the high school tours by non-counselors or fake registration to the seminars.

3.3. Reliability vs Complexity

We used an event repository and PostgreSQL queries to fetch and update the data to update the event schedules correctly after any change in the event applications object. This approach adds some complexity because we need to filter events based on their status, day, and other criteria required. The complexity comes from handling various edge cases, making sure data is consistent across different statuses and days, and keeping the queries efficient. Despite this, it's necessary to achieve reliable updates in real time. This method increases the complexity of our code, but any changes in event applications are accurately reflected in the schedules.

SubSystem Decomposition Diagram

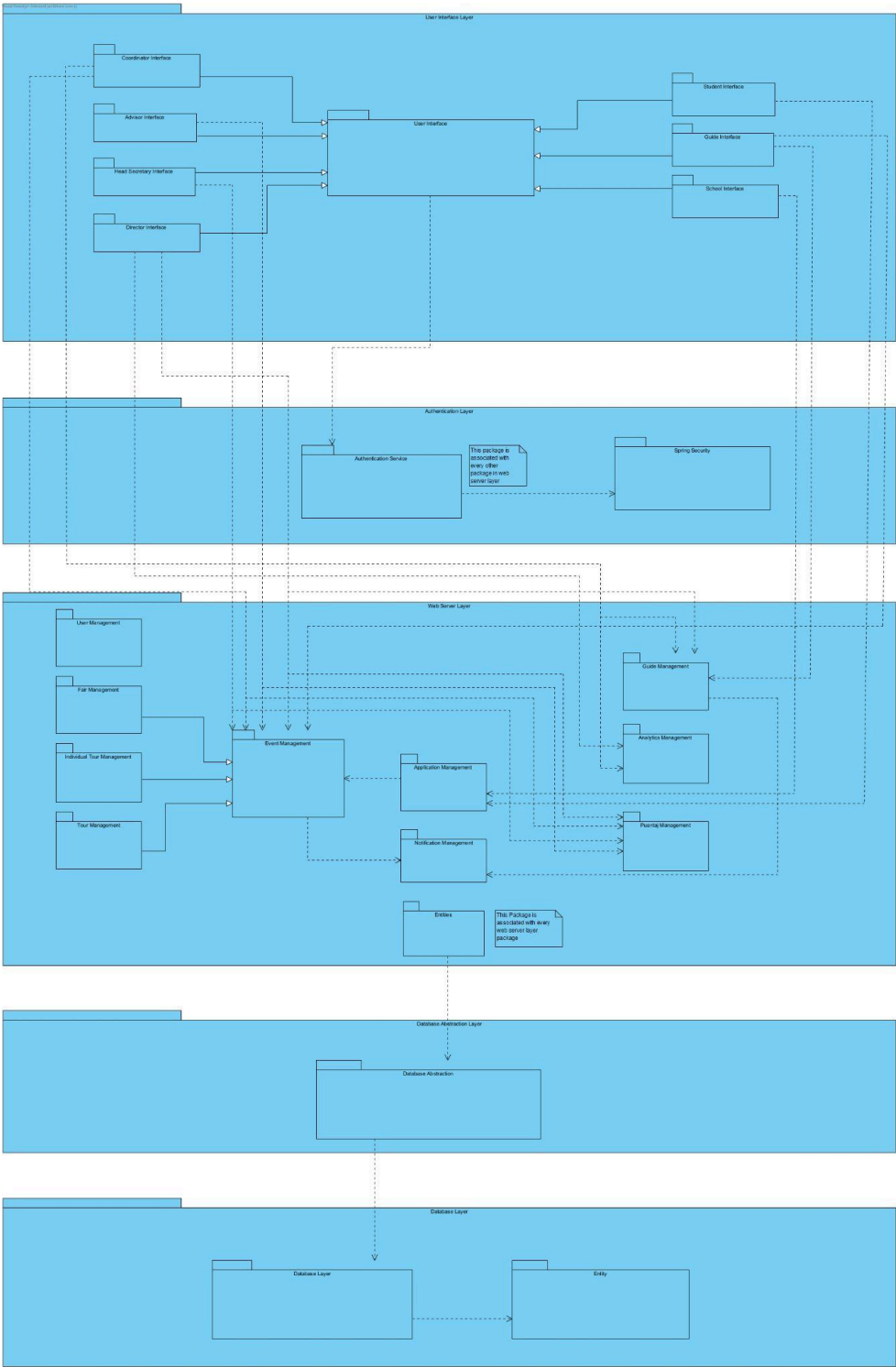


Figure 1. Subsystem Decomposition Diagram